

Ochrona Danych - lab 3 - sprawozdanie

Kody BCH

Jakub Jaśków

25 listopada 2025

Cel ćwiczenia

Celem laboratorium było:

- zapoznanie się z działaniem kodów BCH i ich właściwościami korekcyjnymi,
- uruchomienie programu BCH_P i obserwacja działania kodu BCH(15,7),
- samodzielne zaimplementowanie funkcji obliczania syndromu w programie BCH_ST,
- analiza wykrywania błędów na podstawie syndromu.

Zadanie 1

Treść

Uruchomić program BCH_P i prześledzić jego działanie. Zwrócić uwagę na możliwości korekcyjne kodu dla wybranych wektorów wiadomości oraz dla różnych postaci wektora błędów. Program implementuje kod BCH o parametrach:

$$(n, k) = (15, 7), \quad r = 8, \quad d = 5, \quad t = 2.$$

Opis kodu BCH(15,7)

Kod BCH(15,7) posiada:

- $n = 15$ – długość słowa kodowego,
- $k = 7$ – liczbę bitów informacji,
- $r = 8$ – liczbę bitów kontrolnych,
- $t = 2$ – zdolność korekcji do 2 błędów.

Kodowanie polega na podzieleniu wielomianu informacyjnego przez wielomian generujący $g(x)$ oraz dopisaniu reszty jako części kontrolnej.

Obserwacje

Podczas uruchomienia programu zaobserwowano:

- kod poprawnie wykrywa i koryguje pojedynczy błąd,
- kod jest w stanie korygować dwa błędy ($t = 2$),
- przy trzech błędach syndrom jest różny od zera, lecz kod może nie być poprawnie skorygowany,
- istnieją przypadki szczególne, w których błędy rozkładają się w taki sposób, że odebrane słowo należy do przestrzeni kodowej – wtedy syndrom wynosi 0.

Wnioski:

- Syndrom równy zero nie gwarantuje braku błędów, oznacza jedynie, że odebrane słowo spełnia warunki kodu.
- Liczba błędów przekraczająca zdolność korekcyjną kodu prowadzi do błędnego dekodowania.

Zadanie 2

Treść

Napisać własną funkcję obliczania syndromu wektora kodowego i dołączyć ją do programu BCH_ST. Uruchomić program i zweryfikować poprawność działania.

Idea działania syndromu

Syndrom jest resztą z dzielenia wielomianu słowa odebranego przez wielomian generujący $g(x)$. Obliczenia prowadzone są nad ciałem $GF(2)$, co oznacza, że operacja dodawania odpowiada operacji XOR.

Jeżeli:

$$S = 0,$$

to słowo należy do kodu. W przeciwnym przypadku błąd został wykryty.

Implementacja

W programie zaimplementowano następującą funkcję obliczania syndromu:

```
void Calculate_Syndrom(vector<int> &slovo, vector<int> &syndrom)
{
    vector<int> tmp = slovo;

    for (int i = 0; i <= n - r; i++)
    {
```

```

    if (tmp[i] == 1)
    {
        for (int j = 0; j <= r; j++)
            tmp[i + j] ^= wg[j];
    }

    for (int i = 0; i < r; i++)
        syndrom[i] = tmp[n - r + 1 + i];
}

```

Przypadki testowe i wyniki

Przypadek 1: dwa błędy, możliwe do skorygowania

W pierwszym teście wprowadzono dwa błędy w pozycjach tak dobranych, aby mieściły się w zakresie możliwości korekcyjnych kodu (maksymalnie dwa błędy). Algorytm poprawnie wykrył i skorygował błędy.

Przypadek 2: dwa błędy, ale zbyt duża odległość między nimi

W drugim teście wprowadzono również dwa błędy, lecz ich pozycje były od siebie zbyt oddalone. Błędy nie „mieszczą się” w części kontrolnej wektora kodowego w sposób umożliwiający poprawną korekcję. Algorytm wykrywa błąd (syndrom jest różny od zera), ale dekodowanie jest niepoprawne.

Przypadek 3: trzy błędy przy maksymalnej korekcji dwóch błędów

W trzecim teście wprowadzono trzy błędy, podczas gdy kod ma zdolność korekcji $t = 2$. Algorytm wykrywa błąd (syndrom różny od zera), jednak nie jest w stanie poprawnie odtworzyć pierwotnego słowa. W praktyce prowadzi to do błędnego dekodowania.

Wnioski końcowe

- Funkcja obliczania syndromu poprawnie wykrywa błędy w słowie kodowym.
- Kod BCH(15,7) umożliwia korekcję do dwóch błędów i zapewnia wysoką niezawodność transmisji, pod warunkiem że liczba błędów nie przekracza zdolności korekcyjnej.
- Przy zbyt dużej odległości między błędami lub przy trzech błędach algorytm nadal wykrywa błąd (syndrom $\neq 0$), ale nie jest w stanie poprawnie odtworzyć pierwotnej wiadomości.
- Laboratorium pozwoliło prześledzić praktyczne działanie kodów korekcyjnych oraz wpływ liczby i rozkładu błędów na wyniki dekodowania.

```

mango@T14:~/sem2/Ochrona_Danych/lab3$ ./a.out
----- Kod BCH -----

Podaj ciag do wyslania ( 7 znakow '0' lub '1'): 0000000

Zakodowany ciag (wektor kodowy n = 15): 000000000000000

Podaj blad wprowadzony do wektora kodowego: 110000000000000

Odebrany ciag: 110000000000000

Syndrom odebranego ciagu: 10011100
Waga syndromu: 4

W odebranym ciagu sa bledy - rozpoczynam korekcje

Przesuniety ciag: 011000000000000
Syndrom ciagu odebranego: 01001110
Waga syndromu: 4
<Enter>
Przesuniety ciag: 001100000000000
Syndrom ciagu odebranego: 00100111
Waga syndromu: 4
<Enter>

Przesuniety ciag: 000110000000000
Syndrom ciagu odebranego: 11110111
Waga syndromu: 7
<Enter>

Przesuniety ciag: 000011000000000
Syndrom ciagu odebranego: 10010101
Waga syndromu: 4
<Enter>

Przesuniety ciag: 000001100000000
Syndrom ciagu odebranego: 10100010
Waga syndromu: 3
<Enter>

Przesuniety ciag: 000000110000000
Syndrom ciagu odebranego: 01010001
Waga syndromu: 3
<Enter>

Przesuniety ciag: 000000011000000
Syndrom ciagu odebranego: 11000000
Waga syndromu: 2
<Enter>

Koniec procesu korekcji.
Skorygowany ciag: 000000000000000
Nadany ciag: 000000000000000

Koniec programu!
<Enter>

```

Rysunek 1: Przypadek 1 – dwa błędy w słowie kodowym, poprawnie skorygowane przez algorytm.

```

mango@T14:~/sem2/Ochrona_Danych/lab3$ ./a.out
----- Kod BCH -----

Podaj ciag do wyslania ( 7 znakow '0' lub '1'): 0000000

Zakodowany ciag (wektor kodowy n = 15): 000000000000000
[ ]
Podaj blad wprowadzony do wektora kodowego: 100000000000010

Odebrany ciag: 100000000000010

Syndrom odebranego ciagu: 11101010
Waga syndromu: 5

W odebranym ciagu sa bledy - rozpoczynam korekcje

Przesuniety ciag: 010000000000001
Syndrom ciagu odebranego: 01110101
Waga syndromu: 5
<Enter>
Przesuniety ciag: 101000000000000
Syndrom ciagu odebranego: 11010010
Waga syndromu: 4
<Enter>

Przesuniety ciag: 010100000000000
Syndrom ciagu odebranego: 01101001
Waga syndromu: 4
<Enter>

Przesuniety ciag: 001010000000000
Syndrom ciagu odebranego: 11011100
Waga syndromu: 5
<Enter>

Przesuniety ciag: 000101000000000
Syndrom ciagu odebranego: 01101110
Waga syndromu: 5
<Enter>

Przesuniety ciag: 000010100000000
Syndrom ciagu odebranego: 00110111
Waga syndromu: 5
<Enter>

Przesuniety ciag: 000001010000000
Syndrom ciagu odebranego: 11110011
Waga syndromu: 6
<Enter>

Przesuniety ciag: 000000101000000
Syndrom ciagu odebranego: 10010001
Waga syndromu: 3
<Enter>

Ciagu nie da sie skorygowac!

Koniec programu!
<Enter>

```

Rysunek 2: Przypadek 2 – dwa błędy oddalone od siebie na tyle, że nie mogą zostać poprawnie skorygowane.

```

mango@T14:~/sem2/Ochrona_Danych/lab3$ ./a.out
----- Kod BCH -----

Podaj ciąg do wysłania ( 7 znaków '0' lub '1'): 00000000

Zakodowany ciąg (wektor kodowy n = 15): 0000000000000000

Podaj błąd wprowadzony do wektora kodowego: 1011000000000000

Liczba błędów 3 przekracza zdolność korekcyjną kodu = 2
Możliwe błędne działanie algorytmu uproszczonego dekodowania
[]
Odebrany ciąg: 1011000000000000

Syndrom odebranego ciągu: 11001111
Waga syndromu: 6

W odebranym ciągu są błędy - rozpoczynam korekcję

Przesunięty ciąg: 0101100000000000
Syndrom ciągu odebranego: 10001111
Waga syndromu: 5
<Enter>
Przesunięty ciąg: 0010110000000000
Syndrom ciągu odebranego: 10101111
Waga syndromu: 6
<Enter>

Przesunięty ciąg: 0001011000000000
Syndrom ciągu odebranego: 10111111
Waga syndromu: 7
<Enter>

Przesunięty ciąg: 0000101100000000
Syndrom ciągu odebranego: 10110111
Waga syndromu: 6
<Enter>

Przesunięty ciąg: 0000010110000000
Syndrom ciągu odebranego: 10110011
Waga syndromu: 5
<Enter>

Przesunięty ciąg: 0000001011000000
Syndrom ciągu odebranego: 10110001
Waga syndromu: 4
<Enter>

Przesunięty ciąg: 0000000101100000
Syndrom ciągu odebranego: 10110000
Waga syndromu: 3
<Enter>

Przesunięty ciąg: 0000000010110000
Syndrom ciągu odebranego: 01011000
Waga syndromu: 3
<Enter>

Ciagu nie da się skorygować!

Koniec programu!
<Enter>

```

Rysunek 3: Przypadek 3 – trzy błędy w słowie kodowym przy możliwościach korekcji maksymalnie dwóch błędów.